

TEMA N° 4



TEMA 4 SUCESIONES Y SERIES NUMÉRICAS

TEMA 4. Sucesiones y series numéricas

4.1. Concepto, clasificación y gráfica de una sucesión.

Definición
Define un concepto de secuencia necesaria y que sea extendida en gráfica una sucesión.
 $\{1, 3, 5, 7, \dots\}$

Clasificación
Creciente Decreciente Oscilante

$\{ \dots, \dots \}$

a_n

n

4.2. Fórmulas y aplicación de la progresión aritmética.

Término general
 $a_n = a_1 + (n - 1)d$
 a_1 = primer término
 n = número de términos
 d = diferencia

Suma de los n primeros términos
 $S_n = \frac{n}{2}(a_1 + a_n)$

Aplicación
 $S_n = \frac{n}{2}(a_1 + a_n)$

5 10 15 20

4.3. Fórmulas y aplicación de la progresión geométrica.

Término general
 $a_n = a_1 \cdot r^{n-1}$
 a_1 = primer término
 n = número de términos
 r = razón

Suma de los n primeros términos
 $S_n = a_1 \left(\frac{r^n - 1}{r - 1} \right)$

Aplicación
 $S_n = a_1 \left(\frac{r^n - 1}{r - 1} \right)$
1, 2, 4, 8, ... e^n

4.4. Concepto de límite de una sucesión.

$\left\{ \frac{1}{n} \right\} \rightarrow 0$

$\lim_{n \rightarrow \infty} a_n = L$

a_n

L

n

4.5. Aplicación de sucesiones en algoritmos computacionales.

$\{1, 3, 5, 7, \dots\}$
 $> \{1, 3, 5, 7, \dots\}$
 $O(n)$

$\{1, 1, 2, 3, 5, \dots\}$
 $\text{fib} \{1, 1, 2, 3, 5, \dots\}$
 $O(2^n)$

Cloud, Database, Network icons

EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender y clasificar** las sucesiones numéricas, identificando sus regularidades y comportamientos gráficos mediante el análisis de datos reales aplicados al entorno tecnológico.
2. **Dominar y aplicar** las fórmulas fundamentales de las progresiones aritméticas y geométricas para resolver problemas críticos de infraestructura informática, cálculo de almacenamiento y expansión de redes de datos.
3. **Analizar el comportamiento a largo plazo** de una sucesión mediante el concepto de límite y convergencia, implementando estos modelos matemáticos dentro de algoritmos computacionales eficientes empleando el lenguaje de programación Python.



PRÁCTICA



¿Cómo puede un Técnico en Sistemas Informáticos predecir con precisión matemática si el almacenamiento de un servidor web colapsará antes del fin de año, o cuántos metros de cable UTP se necesitan para conectar de forma óptima los laboratorios de un nuevo módulo educativo?

Imaginemos la siguiente situación real en nuestra región. El Centro de Educación Alternativa (CEA) Herman Ortiz Camargo, ubicado en el Barrio Saó de la población de Pailón (a unas 5 cuadras del centro urbano), acaba de inaugurar un moderno módulo educativo propio. Este complejo cuenta con laboratorios informáticos avanzados que brindan conectividad y servicios digitales a los estudiantes que asisten desde diversos puntos como el Barrio Central Fátima, el Barrio 24 de Septiembre y el Barrio Residencial Norte.

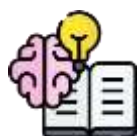
El servidor central de este módulo almacena los proyectos de desarrollo de software y bases de datos de toda la comunidad estudiantil. Durante la primera semana de clases, se registra que el sistema consume un volumen de almacenamiento inicial. Al iniciar la segunda semana, debido a la carga de copias de seguridad de las prácticas de desarrollo web de los estudiantes que provienen del Barrio 27 de Mayo y Barrio San Antonio, el almacenamiento ocupado se incrementa en una cantidad fija respecto a la semana anterior. Este mismo incremento exacto se repite de manera constante al inicio de la tercera y cuarta semana. El programador a cargo necesita determinar cuántos Gigabytes de información se estarán acumulando de forma acumulativa en la semana número 24 para evitar una saturación del sistema.

De manera paralela, un grupo de estudiantes del CEA decide implementar una red comunitaria inalámbrica para conectar una serie de nodos de soporte técnico entre los comercios del Barrio Central Fátima y los campos deportivos del Barrio 24 de Septiembre. El nodo principal se configura en la iglesia central. Debido a la retransmisión de señales y al protocolo de enrutamiento dinámico configurado, cada repetidor adicional que se instala duplica exactamente el número de paquetes de datos que procesó el nodo anterior en un intervalo de un milisegundo. Si el primer equipo procesa una cantidad base de paquetes, el Técnico debe calcular con urgencia



la carga masiva que soportará el décimo repetidor para evitar que la memoria RAM de los routers colapse por desbordamiento de búfer.

Sin recurrir aún a definiciones teóricas ni fórmulas abstractas, este escenario nos demuestra que la informática no opera con datos caóticos. Existe un orden secuencial, un patrón numérico predecible detrás del tráfico de red, del consumo de discos duros y del rendimiento de los procesadores. Analizar estos comportamientos de forma práctica es lo que nos permitirá automatizar soluciones eficientes en nuestro entorno profesional.



TEORÍA



4.1. CONCEPTO, CLASIFICACIÓN Y GRÁFICA DE UNA SUCESIÓN

En el ámbito de la informática, una **sucesión** se define matemáticamente como una función cuyo dominio es el conjunto de los números enteros positivos ($n = 1, 2, 3, \dots$). En palabras más sencillas para un Técnico, una sucesión es una lista ordenada e infinita de números o elementos de datos, donde cada miembro ocupa una posición específica y sigue una regla o patrón lógico predeterminado. Cada elemento de la sucesión se denomina **término** y se representa genéricamente como a_n , donde el subíndice n indica la posición o el índice del término dentro de la lista.

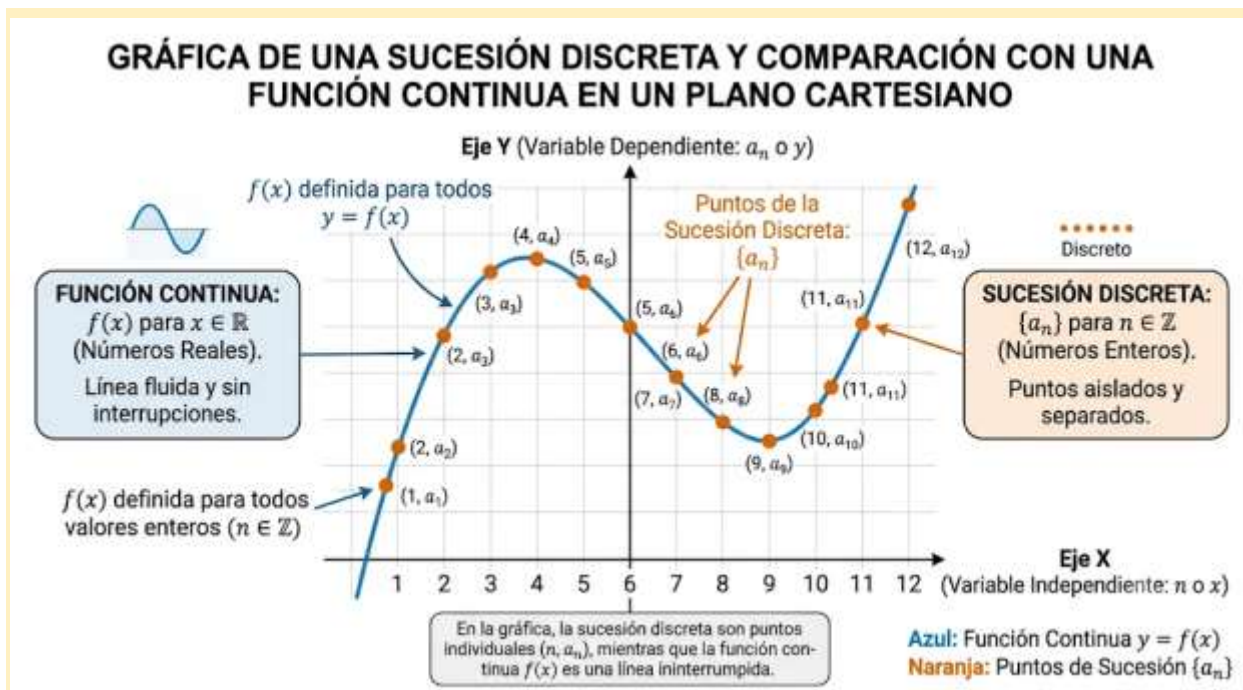
Las sucesiones se pueden clasificar principalmente bajo tres criterios fundamentales:

1. **Según el número de términos:** Pueden ser *finitas* (tienen un número limitado de elementos, como los puertos físicos disponibles en un switch de red de 24 bocas del laboratorio del Barrio Saó) o *infinitas* (se extienden indefinidamente, como el flujo continuo de marcas de tiempo en un archivo de registro de eventos de un servidor).
2. **Según su monotonía:** Una sucesión es *creciente* si cada término es mayor o igual que el anterior ($a_{(n+1)} \geq a_n$), como el contador de visitas acumuladas del sitio web institucional del CEA. Es *decreciente* si cada término es menor o igual que el anterior ($a_{(n+1)} \leq a_n$), como el porcentaje de carga de la batería de una estación IoT basada en ESP32 instalada en el Barrio Jardines de Pailón.



3. **Según su acotación:** Una sucesión está *acotada superiormente* si existe un número real que ningún término de la sucesión puede superar (por ejemplo, la capacidad máxima de memoria RAM de un servidor en gigabytes). Está *acotada inferiormente* si existe un valor mínimo que los términos no pueden sobrepasar hacia abajo (como el valor cero en un contador de descargas de software).

La representación gráfica de una sucesión difiere de las funciones continuas tradicionales de las matemáticas de secundaria. Al tener un dominio discreto (números enteros), la gráfica de una sucesión consiste únicamente en **puntos aislados** en el plano cartesiano, donde el eje horizontal (X) representa el índice de la posición n y el eje vertical (Y) representa el valor del término a_n .



4.2. FÓRMULAS Y APLICACIÓN DE LA PROGRESIÓN ARITMÉTICA

Una **progresión aritmética (PA)** es un tipo especial de sucesión numérica en la cual la diferencia entre cualquier par de términos consecutivos es siempre una cantidad constante. A esta constante se le denomina **diferencia común** y se la representa habitualmente con la letra d .

Para calcular cualquier término en una posición n (llamado término general o a_n) sin necesidad de escribir toda la lista de datos anterior, un Técnico utiliza la fórmula fundamental:



$$a_n = a_1 + (n - 1) \cdot d$$

Donde:

1. a_n : Es el valor del término en la posición n que deseamos encontrar.
2. a_1 : Es el primer término de la progresión (el valor inicial).
3. n : Es la posición o número del término en la secuencia.
4. d : Es la diferencia común entre los términos.

Cuando se requiere calcular el impacto acumulado total (por ejemplo, el total de cables utilizados, costos financieros acumulados o almacenamiento total consumido), se utiliza la fórmula de la **suma de los primeros n términos** (S_n):

$$S_n = (n \cdot (a_1 + a_n)) / (2)$$

Caso de Uso Práctico en Pailón:

El departamento de soporte técnico del CEA Herman Ortiz Camargo está diseñando una red de cableado estructurado para interconectar computadoras en el bloque administrativo del Barrio Saó. En el primer rack de comunicaciones se acomodan 12 cables de red ($a_1 = 12$). Debido a la distribución física de las oficinas hacia el fondo del pasillo, cada rack subsiguiente requiere exactamente 4 cables más que el anterior para compensar las distancias de canalización ($d = 4$). ¿Cuántos cables se requerirán para el rack número 15 ($a_{(15)}$)?

Aplicando la fórmula del término general de forma analítica:

$$a_{(15)} = 12 + (15 - 1) \cdot 4$$

$$a_{(15)} = 12 + (14) \cdot 4$$

$$a_{(15)} = 12 + 56 = 68 \text{ cables}$$

Para el decimoquinto rack se necesitarán exactamente 68 cables de red. Si quisiéramos saber el total absoluto de cables instalados en los 15 racks combinados, aplicaríamos la fórmula de la suma:

$$S_{(15)} = (15 \cdot (12 + 68)) / (2) = (15 \cdot 80) / (2) = 15 \cdot 40 = 600 \text{ cables}$$



4.3. FÓRMULAS Y APLICACIÓN DE LA PROGRESIÓN GEOMÉTRICA

Una **progresión geométrica (PG)** es una sucesión numérica en la cual cada término, a partir del segundo, se obtiene multiplicando el término anterior por una cantidad constante no nula llamada **razón común**, representada por la letra r . A diferencia de la progresión aritmética donde se suma, en la geométrica los valores sufren una multiplicación exponencial, lo que se traduce en un crecimiento o decrecimiento sumamente acelerado de los datos.

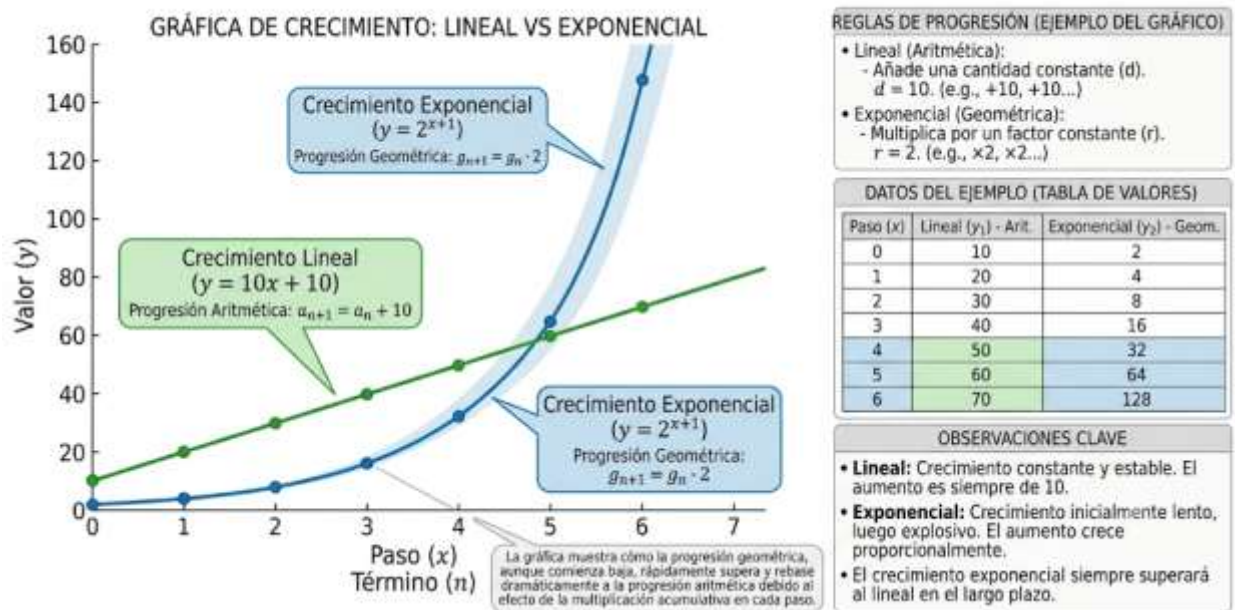
La fórmula matemática para determinar el término general a_n de una progresión geométrica es:

$$a_n = a_1 \cdot r^{(n - 1)}$$

Para calcular la suma acumulada de los primeros n términos en una progresión geométrica, se aplica la siguiente expresión matemática (válida para cuando $r \neq 1$):

$$S_n = (a_1 \cdot (r^n - 1)) / (r - 1)$$

COMPARACIÓN DE CRECIMIENTO: LINEAL (ARITMÉTICA) VS EXPONENCIAL (GEOMÉTRICA)



Caso de Uso Práctico en Pailón:

Un Técnico configura un sistema automatizado de alertas de ciberseguridad para una pequeña empresa distribuidora en el Barrio Central Fátima. Al detectarse un intento de intrusión en el sistema de facturación, el script de seguridad envía de inmediato 3 alertas por correo electrónico en el primer minuto ($a_1 = 3$). Debido a la replicación automatizada del protocolo malicioso, el



número de solicitudes de alerta generadas se triplica cada minuto subsiguiente ($r = 3$). ¿Cuántas alertas inundarán el servidor de correo exactamente al minuto 6 (a_6)?

Calculamos el término empleando la fórmula de la PG:

$$a_6 = 3 \cdot 3^{(6 - 1)}$$

$$a_6 = 3 \cdot 3^5$$

$$a_6 = 3 \cdot 243 = 729 \text{ alertas}$$

En el minuto 6 se generarán de golpe 729 alertas, lo que demuestra la velocidad de crecimiento de una PG y la importancia de que el Técnico programe límites preventivos en los bucles de código para evitar ataques de Denegación de Servicio (DoS).

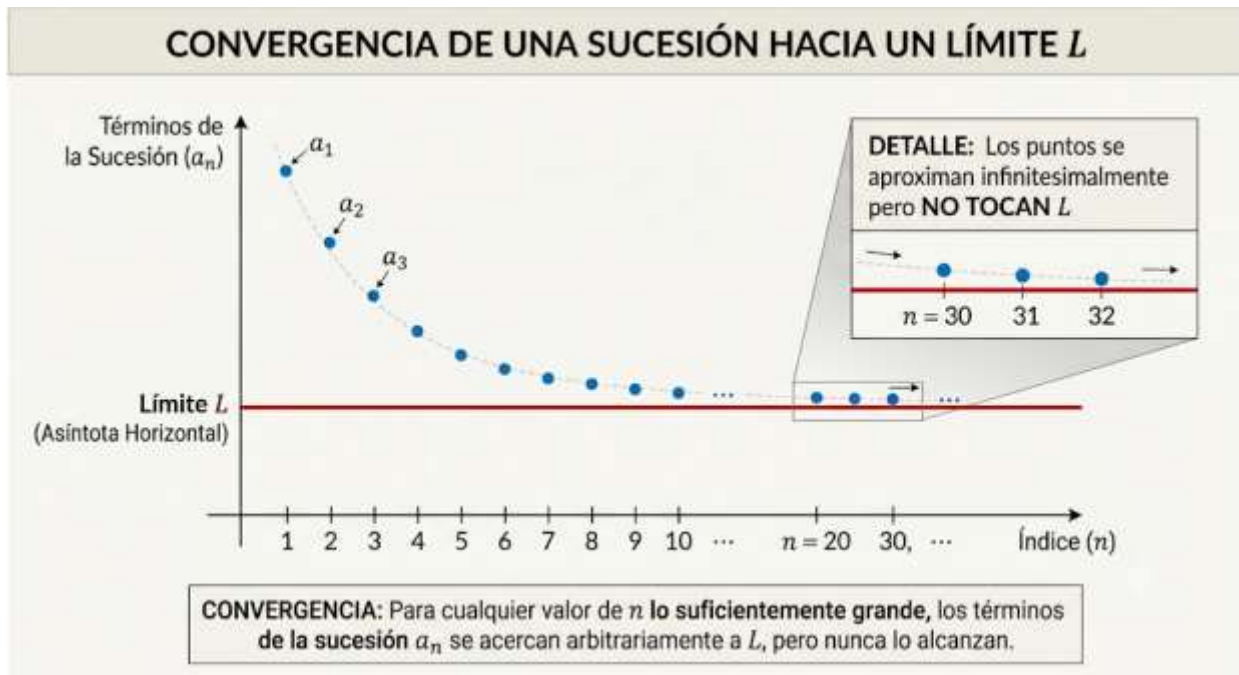
4.4. CONCEPTO DE LÍMITE DE UNA SUCESIÓN

El concepto de **límite de una sucesión** es uno de los pilares del análisis matemático y tiene una aplicación directa en la evaluación del rendimiento y estabilidad de sistemas informáticos. El límite de una sucesión analiza el comportamiento o la tendencia de los términos a_n a medida que el índice n crece de manera indefinida hacia el infinito positivo ($n \rightarrow \infty$).

Formalmente, si los términos de una sucesión se aproximan cada vez más a un número real fijo L conforme avanzamos en la secuencia, decimos que la sucesión es **convergente** y que su límite es L . Esto se escribe matemáticamente de la siguiente manera:

$$\lim_{(n \rightarrow \infty)} a_n = L$$

Si una sucesión no tiende a ningún valor numérico fijo (es decir, crece indefinidamente hacia el infinito o bien oscila de manera caótica entre múltiples valores sin estabilizarse), se la clasifica como una sucesión **divergente**.



Caso de Uso Práctico en Pailón:

En el Barrio Residencial Norte, los Técnicos del CEA instalaron un microservidor local para gestionar la descarga de materiales educativos pesados. Para optimizar el ancho de banda disponible, programaron un algoritmo de compresión dinámica cuyo factor de latencia residual en milisegundos tras cada paquete procesado (n) está determinado por la fórmula analítica de la sucesión:

$$a_n = (4n + 2) / (2n)$$

Para determinar la estabilidad del servidor a largo plazo cuando la cantidad de datos procesados sea inmensa, calculamos el límite cuando n tiende al infinito:

$$\lim_{(n \rightarrow \infty)} (4n + 2) / (2n) = \lim_{(n \rightarrow \infty)} \left((4n) / (2n) + (2) / (2n) \right) = \lim_{(n \rightarrow \infty)} \left(2 + (1) / (n) \right)$$

Sabemos matemáticamente que a medida que n se vuelve infinitamente grande, la fracción $(1) / (n)$ se vuelve tan pequeña que su valor tiende a 0. Por lo tanto:

$$2 + 0 = 2 \text{ milisegundos}$$



Este resultado nos indica con certeza técnica que la latencia residual del servidor jamás se saldrá de control; se estabilizará de forma convergente exactamente en 2 milisegundos, garantizando la estabilidad operativa del servicio informático en el Barrio Residencial Norte.

4.5. APLICACIÓN DE SUCESIONES EN ALGORITMOS COMPUTACIONALES

En las ciencias de la computación, las sucesiones numéricas no se quedan plasmadas únicamente en un papel; se convierten en instrucciones de código ejecutable. Las estructuras lógicas de repetición (for y while) son la traducción directa de una sucesión matemática al entorno de desarrollo de software. Un arreglo de datos (Array o Lista) indexado desde 0 o 1 representa el almacenamiento secuencial de los términos a_n en celdas de memoria física direccionables de la computadora.

A continuación, analizaremos cómo se estructuran y generan de forma algorítmica las progresiones aritméticas y geométricas utilizando código fuente en lenguaje Python, herramienta clave para automatizar tareas informáticas:

Python

```
# Ejemplo de Generación Algorítmica de una Progresión Aritmética
# Contexto: Planificación de direccionamiento IP para switches en Barrio Saódef
generar_progresion_aritmetica(inicial, diferencia, cantidad_terminos): #
Inicializamos una lista vacía para almacenar los términos de la sucesión
sucesion_ip = [] # El bucle '
for
' actúa como el índice matemático '
n
' de nuestra secuencia for n in range(1, cantidad_terminos + 1): #
Aplicamos de forma algorítmica la fórmula matemática fundamental:  $a_n = a_1 + (n - 1) * d$ 
termino = inicial + (n - 1) * diferencia # Añadimos el valor
calculado a nuestra estructura de datos indexada
sucesion_ip.append(termino) # Retornamos la estructura completa con
la sucesión de datos generada return sucesion_ip
# Ejecución del script simulando los identificadores de VLAN de red en el módulo
educativovlan_inicial = 10 # Valor inicial a incremento_vlan = 5 #
Diferencia común dtotal_switches = 8 # Cantidad de posiciones a evaluar
nresultado_pa = generar_progresion_aritmetica(vlan_inicial, incremento_vlan,
total_switches)
print("
Sucesión Aritmética de Red:", resultado_pa)
# Salida esperada: [10, 15, 20, 25, 30, 35, 40, 45]
```



Del mismo modo, podemos evaluar código estructurado para modelar comportamientos geométricos rápidos, de modo que el software tome decisiones automatizadas si detecta que una variable crítica crece a tasas multiplicativas elevadas.

Python

```
# Ejemplo de Generación Algorítmica de una Progresión Geométrica
# Contexto: Escalabilidad de peticiones concurrentes en el servidor del Barrio Central Fátima
def generar_progresion_geometrica(inicial, razon, cantidad_terminos):
    # Estructura de almacenamiento discreto de datos
    registro_peticiones = []
    # Bucle iterativo que emula las posiciones n de la malla matemática
    for n in range(1, cantidad_terminos + 1):
        # Implementación algorítmica de la ecuación: an = a1 * (r ** (n-1))
        termino = inicial * (razon ** (n - 1))
        # Registro posicional paso a paso en el arreglo de memoria
        registro_peticiones.append(termino)
    return registro_peticiones

# Simulación de solicitudes de conexión concurrentes en hora pico
peticiones_base = 2
factor_multiplicador = 4
razon_geometrica = 4
intervalos_tiempo = 5
cantidad_periodos = 5

resultado_pg = generar_progresion_geometrica(peticiones_base, factor_multiplicador, intervalos_tiempo)
print("Sucesión Geométrica de Peticiones:", resultado_pg)
# Salida esperada: [2, 8, 32, 128, 512]
```

4.6. AVANZADO: SUCESSIONES COMPLEJAS Y LA SECUENCIA DE FIBONACCI EN ALGORITMOS DE OPTIMIZACIÓN DE BÚSQUEDA

Más allá de los incrementos lineales o multiplicativos directos de las progresiones básicas, existen sucesiones numéricas más complejas definidas de forma recursiva. En una sucesión recursiva, el valor de un término depende directamente de los términos que le precedieron inmediatamente en la secuencia. La más famosa y utilizada de estas estructuras en la ingeniería de sistemas es la **Sucesión de Fibonacci**, definida matemáticamente mediante las condiciones iniciales $F_0 = 0$, $F_1 = 1$, y la ecuación recursiva general aplicable a partir de la tercera posición:

$$F_n = F_{(n-1)} + F_{(n-2)}$$

Dando origen a la famosa cadena numérica: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Para un Técnico en Sistemas Informáticos, esta secuencia no es un simple ejercicio de curiosidad matemática; representa la base operativa de algoritmos altamente eficientes como la **Búsqueda**



de Fibonacci (Fibonacci Search). A diferencia de la búsqueda secuencial ordinaria (que revisa elemento por elemento gastando muchos recursos de procesamiento) o la búsqueda binaria tradicional (que divide un arreglo estrictamente a la mitad), la Búsqueda de Fibonacci utiliza los números de la sucesión para calcular los índices de examen de un arreglo ordenado de datos.

Este enfoque reduce drásticamente las operaciones de comparación en la memoria caché del procesador informático. Es utilizado ampliamente en sistemas embebidos de bajo consumo de recursos y bases de datos locales indexadas que se despliegan en redes distribuidas, optimizando los tiempos de respuesta en consultas de inventario o registros de usuarios distribuidos por los barrios de la población de Pailón.

4.7. AVANZADO: SERIES NUMÉRICAS APLICADAS A LA OPTIMIZACIÓN DE ALMACENAMIENTO Y COMPRESIÓN DE DATOS

Es de vital importancia aclarar la clara distinción conceptual que existe entre una sucesión y una serie. Mientras que una sucesión es el conjunto ordenado de elementos numéricos individuales, una **serie numérica** representa la expresión matemática de la suma acumulada de todos los términos de dicha sucesión. Si la sucesión es infinita, estamos ante una serie infinita, denotada formalmente utilizando el símbolo de sumatoria de la siguiente manera:

$$\sum_{n=1}^{\infty} a_n = a_1 + a_2 + a_3 + \dots + a_n + \dots$$

En el universo técnico de la informática, las series convergentes geométricas se explotan activamente para el diseño de algoritmos de **compresión de datos** y optimización del almacenamiento en unidades de estado sólido (SSD). Consideremos el espacio físico que ocupa un archivo dentro de un disco de almacenamiento. Si utilizamos un formato de codificación avanzado (como la Codificación de Huffman o algoritmos de compresión sin pérdida basados en series fraccionarias), el tamaño relativo de los datos redundantes disminuye de manera puramente geométrica en cada pasada del compresor de software.

Por ejemplo, si un archivo de base de datos del CEA condensa sus campos repetitivos ocupando primero la mitad del espacio original ((1) / (2)), luego la cuarta parte en la siguiente recursión ((1) / (4)), después la octava parte ((1) / (8)), la suma de estas partes constituye una serie geométrica infinita:



$$\sum_{n=1}^{\infty} \left(\frac{1}{2}\right)^n = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots = 1$$

Esta convergencia matemática exacta a la unidad permite a los desarrolladores estructurar sistemas de archivos inteligentes que fragmentan y empaquetan la información binaria dentro de bloques lógicos del disco sin desperdiciar un solo bit de espacio, logrando almacenar terabytes de registros institucionales dentro de discos físicos compactos instalados en servidores de áreas rurales.

4.8. AVANZADO: EL PAPEL DE LAS SUCESIONES Y SERIES EN EL ENTRENAMIENTO DE MODELOS DE INTELIGENCIA ARTIFICIAL

Hoy en día, la Inteligencia Artificial (IA) y el aprendizaje automático (Machine Learning) guían la automatización industrial y el análisis predictivo. La forma exacta en que una máquina aprende de sus errores para identificar patrones complejos se fundamenta enteramente en el concepto matemático de sucesiones numéricas a través de un algoritmo crítico conocido como **Descenso de Gradiente (Gradient Descent)**.

Cuando una red neuronal artificial es entrenada para reconocer patrones de texto o predecir fallas en la red eléctrica de Pailón, el algoritmo calcula una función de pérdida o error, la cual mide qué tan alejadas están las predicciones matemáticas de la realidad operativa. Para minimizar ese error, el algoritmo actualiza repetidamente los pesos internos del modelo matemático, generando una **sucesión iterativa de parámetros de ajuste** ($w_0, w_1, w_2, \dots, w_t$) regida por la siguiente expresión de recurrencia algorítmica:

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t)$$

Donde:

1. w_t : Representa el estado o valor actual de los pesos de la red en la iteración t .
2. η (Eta): Es la tasa de aprendizaje (Learning Rate), un factor crítico de escala.
3. $\nabla L(w_t)$: Es el gradiente matemático, que indica la dirección del error.

Los valores numéricos secuenciales generados en cada ciclo de entrenamiento (w_t) constituyen una sucesión matemática estricta. El objetivo fundamental del programador o Técnico especializado es configurar la tasa de aprendizaje con tal precisión matemática que esta sucesión



sea rigurosamente **convergente**. El límite de esta sucesión en el infinito debe apuntar con exactitud al valor mínimo global del error de la red. Si la sucesión generada resulta ser divergente debido a un mal diseño lógico, la Inteligencia Artificial jamás aprenderá nada útil, sufriendo desbordamientos de datos numéricos y oscilando caóticamente sin lograr estabilizar sus procesos predictivos.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito	Realidad
Las sucesiones matemáticas solo sirven para resolver ejercicios teóricos abstractos en la pizarra y no se aplican en la vida real de un programador.	Las sucesiones son la base lógica detrás del control de bucles en la programación, optimización de algoritmos de búsqueda, compresión de archivos y la infraestructura de redes informáticas.
Toda sucesión que crece con números cada vez más grandes de manera indefinida no tiene ninguna utilidad práctica dentro de la informática.	Las sucesiones divergentes controladas modelan el crecimiento del tráfico en redes, ayudando a los Técnicos a dimensionar la memoria de servidores antes de colapsar.

❑ Glosario de Términos

1. **Sucesión:** Función matemática cuyo dominio son los enteros positivos, manifestada como una lista infinita y ordenada de términos numéricos que siguen un patrón lógico.
2. **Término General (a_n):** Expresión analítica o fórmula matemática que permite calcular directamente el valor numérico de cualquier elemento de una sucesión dada su posición n .
3. **Convergencia:** Propiedad de una sucesión cuyos términos se van aproximando indefinidamente a un valor único y constante bien definido a medida que el índice de análisis tiende al infinito.
4. **Diferencia Común (d):** Valor constante que se suma al término actual en una progresión aritmética para obtener el valor del siguiente término de la secuencia.



5. **Razón Geométrica (r):** Factor constante por el cual se multiplica el valor del término actual en una progresión geométrica para calcular el término inmediatamente posterior.

□ **Ponte a Prueba**

1. Un script automatizado de copias de seguridad de bases de datos configurado en el servidor del Barrio Saó genera archivos cuyo tamaño sigue una progresión aritmética perfecta. Si el primer archivo de respaldo pesa 15 MB ($a_1 = 15$) y cada día subsiguiente el tamaño aumenta de forma constante en 8 MB ($d = 8$), ¿cuál será el tamaño exacto del archivo de copia de seguridad al cabo de 10 días ($a_{(10)}$)?

1. a) 95 MB
2. b) 87 MB
3. c) 80 MB
4. **Respuesta Correcta: b) 87 MB.** Explicación paso a paso: Empleando la ecuación matemática de la progresión aritmética: $a_{(10)} = 15 + (10 - 1) \cdot 8 \rightarrow a_{(10)} = 15 + (9) \cdot 8 \rightarrow a_{(10)} = 15 + 72 = 87$ MB.

2. Una vulnerabilidad crítica de tipo bucle infinito en un script web desplegado maliciosamente en una red local del Barrio Central Fátima triplica el consumo de sockets de conexión de red abiertos cada segundo. Si en el segundo 1 el sistema arranca con 2 sockets abiertos ($a_1 = 2$), ¿cuántas conexiones en paralelo mantendrá abiertas de forma colosal al llegar al segundo 5 (a_5)?

1. a) 162 conexiones
2. b) 54 conexiones
3. c) 243 conexiones
4. **Respuesta Correcta: a) 162 conexiones.** Explicación paso a paso: Al tratarse de un factor multiplicativo constante, aplicamos la fórmula geométrica: $a_5 = 2 \cdot 3^{(5 - 1)} \rightarrow a_5 = 2 \cdot 3^4 \rightarrow a_5 = 2 \cdot 81 = 162$ conexiones.



3. Si un algoritmo de optimización de búfer de datos para una antena inalámbrica que conecta el Barrio 24 de Septiembre presenta un comportamiento de descarte de paquetes erróneos definido por la sucesión matemática analítica $a_n = (6n - 5) / (3n)$, ¿a qué valor crítico de convergencia se estabilizará el sistema cuando el tráfico tienda al infinito ($n \rightarrow \infty$)?

1. a) 0 paquetes descartados
2. b) 3 paquetes descartados
3. c) 2 paquetes descartados
4. *Respuesta Correcta:* **c) 2 paquetes descartados.** *Explicación paso a paso:* Calculando analíticamente el límite al infinito de la función racional: $\lim_{(n \rightarrow \infty)} (6n - 5) / (3n) = \lim_{(n \rightarrow \infty)} \left((6n) / (3n) - (5) / (3n) \right) = 2 - 0 = 2$. El sistema converge de forma estable a un valor fijo de 2.



VALORACIÓN



El estudio de las sucesiones y series numéricas en el ámbito de los Sistemas Informáticos va mucho más allá de un mero cálculo frío en una pantalla; conlleva una profunda responsabilidad ética, social y medioambiental que todo Técnico debe sopesar detenidamente dentro de su comunidad.

En primer lugar, analicemos el fenómeno del crecimiento de la basura electrónica o **e-waste**. El hardware informático evoluciona a ritmos geométricos acelerados; la capacidad de procesamiento de los procesadores y la demanda de software complejo se duplican de forma exponencial cada cierto tiempo. Este ritmo vertiginoso presiona a las instituciones y empresas locales de la población de Pailón a desechar equipos de computación que aún funcionan perfectamente, dejándolos obsoletos en pocos años. Un Técnico responsable debe aplicar sus conocimientos matemáticos para optimizar el software existente mediante algoritmos eficientes (como los basados en la secuencia de Fibonacci o estructuras lógicas de compresión de datos),



alargando la vida útil del hardware disponible y mitigando el impacto medioambiental del desecho tecnológico en los barrios periféricos como San Expedito o Las Misiones.

En segundo lugar, se debe considerar el impacto ético en la **seguridad y la privacidad de la información**. Los algoritmos modernos de criptografía, encargados de proteger las contraseñas bancarias de los ciudadanos del Barrio Central Fátima y los expedientes académicos confidenciales guardados en el módulo educativo propio del Barrio Saó, se basan en propiedades numéricas de sucesiones complejas y generación de cadenas pseudoaleatorias de números primos. Si un Técnico implementa código repetitivo de forma descuidada, estructurando bucles ineficientes que sigan patrones aritméticos fácilmente predecibles, expone los servidores a ataques de fuerza bruta que pueden comprometer la privacidad y el patrimonio financiero de toda una población.

Finalmente, las decisiones basadas en límites y convergencia de redes de telecomunicaciones juegan un papel de **justicia social y reducción de la brecha digital**. Cuando se diseña la infraestructura de red para un municipio en pleno desarrollo como Pailón, el Técnico debe balancear las tasas de transferencia de datos empleando modelos secuenciales rigurosos. Si el despliegue del ancho de banda se distribuye egoístamente siguiendo progresiones descontroladas que solo benefician al centro comercial formal del Barrio Fátima, se condenará al aislamiento digital a los estudiantes de familias humildes que habitan en zonas más alejadas como el Barrio María Belén, Barrio Ovidio Barberi o Barrio San José. El software y la matemática deben usarse como herramientas de integración equitativa y democrática.



PRODUCCIÓN



RETO FINAL / LABORATORIO: DESARROLLO DE UN SISTEMA PREDICTIVO DE CRECIMIENTO DE TRÁFICO Y CAPACIDAD DE ALMACENAMIENTO EN PYTHON

Objetivo del Laboratorio: Implementar un programa ejecutable en Python que emule, modele y calcule con total precisión matemática el comportamiento predictivo de dos tipos de sucesiones de datos para la infraestructura de servidores del CEA Herman Ortiz Camargo. El software



automatizará los reportes técnicos alertando en qué momento exacto se saturarán las capacidades físicas del sistema.

Instrucciones Paso a Paso para la Computadora:

1. Crea un nuevo archivo de código con el nombre analisis_sucesiones_cea.py dentro de tu entorno de desarrollo o editor de texto favorito (Visual Studio Code, PyCharm o IDLE de Python).
2. Escribe minuciosamente el código fuente provisto a continuación. Asegúrate de respetar los espacios de sangría (indentación) de cuatro espacios exigidos por la sintaxis de Python para los bloques de funciones y bucles iterativos.
3. Lee detalladamente los comentarios explicativos integrados línea por línea en el script de código para comprender el mapeo físico de cada variable y fórmula matemática estudiada.
4. Ejecuta el archivo desde la consola del sistema operativo mediante el comando: python analisis_sucesiones_cea.py.
5. Analiza detalladamente los resultados de las salidas impresas por pantalla y elabora conclusiones de ingeniería sobre cómo las sucesiones permiten tomar decisiones tecnológicas preventivas antes de que ocurran fallos en los sistemas reales.

Código Fuente Completo en Python:

Python

```
# Script de Automatización Matemática: Sistema Predictivo de Almacenamiento e
Infraestructura
# Diseñado para Los Laboratorios informáticos del CEA Herman Ortiz Camargo
(Pailón)
import time # Importamos la librería de tiempo para simular retrasos de
procesamiento de red
def evaluar_servidor_backup_aritmetico(): # Bloque 1:
Simulación de Crecimiento Lineal de Archivos en el Barrio Saó
print("=====")
print("
SISTEMA DE PREDICCIÓN DE ALMACENAMIENTO - RACK CORE (BARRIO SAÓ)")
print("=====")
# Declaración de variables de entrada mapeando la fórmula:  $a_n = a_1 + (n-1)*d$ 
almacenamiento_inicial_mb = 500.0 # Término inicial de la sucesión ( $a_1$ )
```



```

incremento_diario_mb = 150.0      # Diferencia común constante (d)
capacidad_maxima_disco_mb = 3000.0 # Cota superior rígida del sistema físico
dia = 1 # Inicializador del contador posicional indexado (n)    espacio_ocupado
= almacenamiento_inicial_mb # El valor inicial arranca en a1      # Bucle de
control iterativo que calcula secuencialmente el avance de la PA    while
espacio_ocupado <= capacidad_maxima_disco_mb:      # Imprimimos de forma
formateada el estado del almacenamiento en el día actual (posición n)
print(f"[Día {
dia}
] Almacenamiento ocupado en el rack del CEA: {
espacio_ocupado:.2f}
MB")      # Avanzamos al siguiente periodo o día indexado en la
sucesión matemática      dia = dia + 1      # Aplicación directa del
incremento secuencial de la diferencia de la progresión      espacio_ocupado =
almacenamiento_inicial_mb + (dia - 1) * incremento_diario_mb      #
Mensaje de alerta técnica ejecutado cuando la sucesión supera la cota física
permitida    print(f"\nCRÍTICO: ¡Almacenamiento Colapsado! El sistema se satura
por completo en el Día {
dia}
.")    print(f"
Espacio teórico final calculado por sucesión: {
espacio_ocupado:.2f}
MB")
def evaluar_trafico_red_geometrico():    # Bloque 2: Simulación de Crecimiento
Exponencial de Tráfico en Barrio Central Fátima
print("\n=====")
print("
MONITOR DE TRÁFICO EXPONENCIAL DE RED - NODO CENTRAL FÁTIMA")
print("=====")
# Declaración de variables de entrada mapeando la fórmula:  $a_n = a_1 * (r ** (n - 1))$ 
peticiones_iniciales = 5      # Primer término de la progresión
geométrica (a1)    factor_multiplicacion = 2      # Razón común multiplicativa
(r)    intervalos_segundos = 6      # Límite de periodos de tiempo n a simular
# Bucle estructurado '
for
' encargado de recorrer rígidamente los índices n definidos    for segundo in
range(1, intervalos_segundos + 1):      # Implementación matemática del
término general de la progresión geométrica de datos
peticiones_en_segundo = peticiones_iniciales * (factor_multiplicacion **
(segundo - 1))      # Despliegue de los logs de rendimiento de red
para examen técnico de paquetes    print(f"[Segundo {
segundo}
] Peticiones en el Nodo Fátima: {
peticiones_en_segundo}
solicitudes/seg")    time.sleep(0.3) # Simulación de delay de red del
microprocesador en milisegundos      # Cálculo analítico del volumen
acumulado global de datos usando la fórmula de la suma de la PG
total_peticiones_acumuladas = (peticiones_iniciales * ((factor_multiplicacion **
intervalos_segundos) - 1)) / (factor_multiplicacion - 1)    print(f"\nANÁLISIS

```



```
ESTADÍSTICO DE INFRAESTRUCTURA:")    print(f"
Total absoluto de paquetes procesados en {
intervalos_segundos}
segundos: {
total_peticiones_acumuladas:.0f}
peticiones.")
# Punto de entrada principal y ejecución secuencial del programa automatizado
if __name__ == "__main__":    evaluar_servidor_backup_aritmetico() # Invocamos la
                             primera función de control matemático lineal    evaluar_trafico_red_geometrico()
                             # Invocamos la segunda función de control multiplicativo exponencial
```

RECURSOS ADICIONALES

1. **Documentación Oficial de Python sobre Estructuras de Control e Iteradores:** Manual de referencia técnico indispensable para profundizar en la sintaxis de bucles, arreglos y generadores de datos.
Enlace recomendado: <https://docs.python.org/3/tutorial/controlflow.html>
2. **Repositorio GitHub de Algoritmos Computacionales de Búsqueda:** Colección de código abierto donde se puede examinar la implementación real de la Búsqueda de Fibonacci y estructuras de ordenamiento secuencial optimizado de datos de memoria.
Enlace recomendado: <https://github.com/TheAlgorithms/Python>
3. **Guía de Optimización de Sistemas de Archivos y Compresión Numérica en MDN Web Docs:** Recurso académico de nivel profesional centrado en cómo se estructuran las series de asignación lógica de bloques y empaquetamiento binario en entornos web y servidores de almacenamiento de alta disponibilidad.
Enlace recomendado: <https://developer.mozilla.org/es/docs/Web/Performance>